

Linux Programming — Exam Revision Pack

University of Bamenda / IUGET · Level 3 · Course code CESP4205 · Lecturer: Eng. Malobe Lottin Cyrille · 2025–2026

Continuous Assessment format: Time 2H · Answer ALL questions · show commands clearly.
Section A Linux Fundamentals (6) · **Section B** Command Line & File System (8) · **Section C** Permissions & Processes (8) · **Section D** Shell Scripting & Network Admin (8).
Full course (9 topics): Intro to UNIX/Linux · Command line & file system · Filters/Pipes/Redirection/Regex · Bash scripting · C programming · System calls & processes · Security · Network programming (sockets) · GUI overview.

CA Exam — Model Answers

Section A — Linux Fundamentals (6 marks)

Q1a. Define: Linux Kernel · Shell · Linux Distribution [3]

- Linux Kernel** — the privileged **core of the operating system** (created by Linus Torvalds, 1991) that manages processes, memory, devices, the file system and networking, acting as the interface between hardware and software.
- Shell** — a **command interpreter** program that reads the commands you type, interprets them, and executes them by launching programs (e.g. `bash`, `zsh`).
- Linux Distribution** — a complete OS package = **Linux kernel + GNU tools, libraries, a shell, package manager, installer and documentation** (e.g. Ubuntu, Debian, Fedora).

Q1b. Differentiate [2]

i. CLI vs GUI

CLI (Command Line Interface)	GUI (Graphical User Interface)
Text-based; you type commands	Visual windows, icons, menus
Fast, scriptable, low resource use	User-friendly but heavier on resources

ii. Open-source vs Proprietary software

Open-source	Proprietary
Source code freely available to view, modify & redistribute (e.g. Linux)	Source code closed/owned by a vendor; cannot be copied or modified (e.g. Windows)

Q1c. One advantage of Linux in [1]

- Server administration:** high stability and uptime with low resource use, managed remotely via SSH (and it is free/open-source).
- Networking environments:** built-in TCP/IP stack and powerful networking tools — it powers most routers, firewalls and web/DNS servers.

Section B — Command Line & File System (8 marks)

Q2. Write the commands [4]

Task	Command
a. Display current working directory	<code>pwd</code>
b. List all files including hidden	<code>ls -a</code> (or <code>ls -la</code>)
c. Create directory <code>linux_CA</code>	<code>mkdir linux_CA</code>
d. Move into <code>linux_CA</code>	<code>cd linux_CA</code>
e. Create empty file <code>student.txt</code>	<code>touch student.txt</code>
f. Copy <code>student.txt</code> into /tmp	<code>cp student.txt /tmp</code>
g. Rename to <code>result.txt</code>	<code>mv student.txt result.txt</code>
h. Remove <code>result.txt</code>	<code>rm result.txt</code>

Q3. File System Navigation [4]

Path: `/home/admin/projects/linux/unit4` · You are in `/home/admin`.

- **a.** Move to unit4 using a **relative** path: `cd projects/linux/unit4`
- **b.** Move to `/etc/network` using an **absolute** path: `cd /etc/network`
- **c. Absolute vs relative path:** an **absolute path** starts from the root directory `/` and gives the full location regardless of where you are; a **relative path** is interpreted from your **current working directory** (and may use `.` for current and `..` for parent).

Section C — File Permissions & Process Management (8 marks)

Q4. Permission `-rwxr-x---` [8]

a. Meaning of each section:

Section	Symbols	Meaning
File type	<code>-</code>	Regular file (a <code>d</code> here would mean a directory)
Owner (user)	<code>rwx</code>	read, write and execute
Group	<code>r-x</code>	read and execute, but no write
Others	<code>---</code>	no permissions at all

In numeric form this is **750** ($rwx=7$, $r-x=5$, $---=0$).

b. **Commands:** change permissions → `chmod`; change ownership → `chown`.

c. **Specific commands:**

- Give execute permission to **all users** on script.sh: `chmod a+x script.sh`
- Remove write permission from the **group** on script.sh: `chmod g-w script.sh`

Q5. Process Management [4]

a. Function of commands:

Command	Function
<code>ps</code>	Shows a snapshot of currently running processes (with their PIDs)
<code>top</code>	Displays a real-time, dynamic view of processes and CPU/memory usage
<code>kill</code>	Sends a signal to terminate (or signal) a process by its PID

b. Terminate the process with PID 2450: `kill 2450` (force: `kill -9 2450`).

Section D — Shell Scripting & Network Administration (8 marks)

Q6. Shell Scripting Fundamentals [3]

- a. Shell scripting** = writing a sequence of shell commands in a file (a **script**) that the shell executes to **automate tasks**.
- b. Two advantages:** (1) **Automates repetitive tasks**, saving time and reducing human error; (2) scripts are **reusable and consistent**, and can be scheduled (e.g. with `cron`) for unattended administration.

Q7. Network Diagnostic Script — `network_check.sh` [5]

```
#!/bin/bash
# network_check.sh - basic network diagnostics
REPORT="network_report.txt"

{
  echo "==== Hostname ====="
  hostname

  echo "==== IP Address Configuration ====="
  ip addr                # (or: ifconfig)

  echo "==== Connectivity test to 8.8.8.8 ====="
  ping -c 4 8.8.8.8

  echo "==== DNS resolution for google.com ====="
  nslookup google.com    # (or: host google.com / dig google.com)

  echo "==== Default Gateway / Routing table ====="
  ip route                # (or: route -n)
} > "$REPORT"

echo "Network report saved to $REPORT"
```

Marks come from: the **shebang** (`#!/bin/bash`), correct **commands**, and **output redirection** (`{ ... } > file` or `>>` per line).

Core Theory Revision

UNIX & Linux — history and terms

- **UNIX** — family of multiuser, multitasking OSes from **Bell Labs, early 1970s**; promoted portability, modularity, "small tools that do one thing well". Mostly rewritten in **C** (4th edition, 1973). **BSD 4.2 (1983)** added TCP/IP + sockets.
- **GNU** (GNU's Not Unix) — Richard Stallman, **1984**: a free-software project producing tools, compilers and libraries; "free" in the sense of freedom.
- **Linux** — strictly the **kernel** by **Linus Torvalds, 1991** (for his Intel 80386 PC). Kernel + GNU tools = a complete OS (hence "GNU/Linux"). Early distros 1992; Slackware/Debian 1993; Ubuntu 2004.
- **Distribution** — kernel + tools + package manager + installer + docs.

The UNIX philosophy

- **Everything is a file** (regular files, directories, device files like `/dev/sda`, pipes, sockets).
- Small programs that do **one thing well** (`ls`, `cat`, `grep`, `sort`).
- Programs **work together via pipes**: `cat file.txt | grep "error" | sort`.
- **Text** as the universal interface · avoid monolithic programs · transparency & simplicity.

Linux architecture (layered, bottom → top)

Layer	Role
User Applications	Editors, browsers, compilers, custom programs
Shell & Utilities	Command interpreters and tools (Bash, ls, cp, grep, ps)
System Call Interface	Controlled gateway for programs to request kernel services
Kernel	Manages processes, memory, devices, files, scheduling, networking, security
Device Drivers	Kernel components that talk to hardware (disk, NIC, keyboard)
Hardware	CPU, memory, storage, buses, network interfaces

User space vs Kernel space: user space runs ordinary apps with **restricted privilege** (access via system calls) — a crash usually affects only that app. Kernel space has **full privileged control** — a fault there can crash the whole system.

*Terminal vs Shell: the **terminal** is the window you type in; the **shell** (e.g. Bash) is the program inside it that interprets your commands.*

Filesystem Hierarchy Standard (common directories)

Dir	Purpose	Dir	Purpose
<code>/</code>	Root of the whole tree	<code>/var</code>	Variable data: logs, mail, spool
<code>/bin</code>	Essential user commands	<code>/tmp</code>	Temporary files
<code>/sbin</code>	System/admin binaries	<code>/usr</code>	User programs & libraries
<code>/etc</code>	System config files	<code>/dev</code>	Device files
<code>/home</code>	Users' home directories	<code>/proc</code>	Kernel/process info (virtual)
<code>/root</code>	root user's home	<code>/boot</code>	Bootloader & kernel images

Essential commands

Area	Commands
Navigation / files	<code>pwd</code> <code>ls</code> <code>cd</code> <code>mkdir</code> <code>touch</code> <code>cp</code> <code>mv</code> <code>rm</code> <code>cat</code> <code>less</code> <code>find</code>
Permissions / ownership	<code>chmod</code> <code>chown</code> <code>chgrp</code> · symbolic (<code>u/g/o/a</code> + <code>+</code> <code>-</code> <code>=</code> + <code>rxw</code>) or numeric (r=4, w=2, x=1)
Processes	<code>ps</code> <code>top</code> <code>kill</code> <code>killall</code> <code>jobs</code> <code>bg</code> <code>fg</code> · <code>&</code> runs in background
Filters & text (Topic 3)	<code>grep</code> (search) <code>sed</code> (stream edit) <code>awk</code> (field processing) <code>sort</code> <code>cut</code> <code>wc</code> <code>head</code> <code>tail</code>
Redirection / pipes	<code>></code> overwrite · <code>>></code> append · <code><</code> input · <code>2></code> errors · <code> </code> pipe
Networking	<code>hostname</code> <code>ip addr</code> <code>ifconfig</code> <code>ping</code> <code>nslookup</code> <code>dig</code> <code>ip route</code> <code>netstat</code> <code>ss</code>

Bash scripting essentials (Topic 4)

```
#!/bin/bash           # shebang: which interpreter runs the script
name="World"          # variable (no spaces around =)
echo "Hello, $name"    # $ expands a variable

if [ "$1" = "start" ]; then      # $1 = first argument
    echo "starting..."
else
    echo "usage: $0 start"
fi

for f in *.txt; do echo "$f"; done # loop
```

Make a script runnable: `chmod +x script.sh` then run it with `./script.sh`.